



UMassAmherst

Manning College of Information
& Computer Sciences

Programming Methodology

Lab 4: Lists and More Closures

Wednesday September 30, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 3-4 to solve a few exercises.
 - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Reminders

- Homework 4 is due next Monday (10/5) at 11:59 pm.
 - Come to [office hours](#) for help!
- **Midterm 1 is next Wednesday (10/7) 7 - 9 pm**
 - **Keep an eye out for room announcement**
- If you need to miss lab for a valid and documented reason:
 - Fill out the excusal form on Canvas before the lab.
 - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
 - Lab submissions are due at 6:00 pm.
 - The submission counts towards your lab grade.
 - **Today, Gradescope closures exercise also counts!**

Today's Goals

- Practice using lists
- More practice reasoning about closures

Review of Lists

Lists are **recursive**.

Lists are either

- empty, or
- an **element** followed by a **list**

Lists are an **abstract data type** with the following methods:

- `list.isEmpty()`
 - returns **True** if `list` is empty()
 - returns **False** if `list` is `node(data, next)` [element followed by list]
- `list.head()`
- `list.tail()`

Exercise 1: Lists

Given two ordered lists, merge them such that the resulting list is an ordered list (ascending).

```
// merges two ordered lists
```

```
function merge(l1: List<number>, l2: List<number>): List<number>
```

Solution 1: Lists

```
// merges two ordered lists
```

```
function merge(l1: List<number>, l2: List<number>) {  
    if (l1.isEmpty()) return l2;  
    if (l2.isEmpty()) return l1;  
  
    const h1 = l1.head();  
    const h2 = l2.head();  
    if (h1 < h2) return node(h1, merge(l1.tail(), l2));  
    return node(h2, merge(l1, l2.tail()));  
}
```

Review of Closures

A **closure** is a function that remembers variables from the environment where it was created.

```
function makeCounter(): () => number {  
  let count = 1;  
  
  return () => count++;  
}
```

```
const counter = makeCounter();
```

```
counter(); // 1  
counter(); // 2
```

What variable is captured
by the returned function?

count

Review of Closures

A **closure** is a function that remembers variables from the environment where it was created.

```
function makeCounter(): () => number {  
  return () => {  
    let count = 1;  
    return count++;  
  };  
}
```

```
const counter = makeCounter();
```

```
counter(); // ???  
counter(); // ???  
counter(); // ???
```

What are the outputs of
the `counter()` calls?

Review of Closures

A **closure** is a function that remembers variables from the environment where it was created.

```
function makeCounter(): () => number {  
  return () => {  
    let count = 1;  
    return count++;  
  };  
}
```

```
const counter = makeCounter();
```

```
counter(); // 1  
counter(); // 1  
counter(); // 1
```

What are the outputs of the
counter() calls?

count was not declared
outside of the returned
function, so it is reset with
each call.

Exercise 2: Closures

```
function makeCounter(tally: { total: number }) {  
  let count = 1;  
  
  return () => {  
    tally.total++;  
    return count++;  
  };  
}
```

```
const tally = { total: 0 };
```

```
const counter1 = makeCounter(tally);  
const counter2 = makeCounter(tally);
```

```
const x = counter1();  
const y = counter1();  
const z = counter2();
```

Without running the code,
answer the questions
about this snippet on
Gradescope.

Solution 2: Closures

```
function makeCounter(tally: { total: number }) {  
  let count = 1;  
  
  return () => {  
    tally.total++;  
    return count++;  
  };  
}
```

```
const tally = { total: 0 };
```

```
const counter1 = makeCounter(tally);  
const counter2 = makeCounter(tally);
```

```
const x = counter1();  
const y = counter1();  
const z = counter2();
```

What are the values of the following variables?

x = 1, y = 2, z = 1

What is the final value of **tally.total**?

tally.total = 3

What state is shared between **counter1** and **counter2**?

tally is shared by both closures.

What state is separate between **counter1** and **counter2**?

Each closure has its own **count**.

Explain why **makeCounter(tally)()** would not produce the values **y+1** or **z+1**.

Each call to **makeCounter** creates a new closure with a new **count** variable.

Solution 2: Closures

```
function makeCounter(  
  tally: { total: number }  
) {  
  let count = 1;  
  
  return () => {  
    tally.total++;  
    return count++;  
  };  
}  
  
const tally = { total: 0 };  
  
const counter1 = makeCounter(tally);  
const counter2 = makeCounter(tally);  
  
const x = counter1();  
const y = counter1();  
const z = counter2();
```

